

Práctica 1: Programación en radio definida por software GNURadio

Juan David Quiñones Chacon - 2231541
Crisntian Mateo Pinilla Rodriguez - 2215591
Brhandon Felipe Herrera Puerto - 2185601

https://github.com/JuanQ1105/2026_2_CommII_A1_GF

Escuela de Ingenierías Eléctrica, Electrónica y de Telecomunicaciones
Universidad Industrial de Santander

1 de septiembre de 2026

Abstract—This report presents the development and implementation of custom Python blocks within the GNU Radio environment, designed for Software-Defined Radio (SDR) applications. Various digital signal processing algorithms were implemented using a collaborative workflow and version control via Git and GitHub. These include accumulator and differentiator blocks, as well as modules for calculating statistical metrics in real time. The system's performance was evaluated under variable sampling rates and noisy environments, validating theoretical digital communication models. Results demonstrate the efficiency of programmable blocks for signal characterization and robust experimentation in SDR platforms.

Index Terms—SDR, GNU Radio, Python, Signal Processing, Accumulator, Differentiator

I Introducción

La radio definida por software (SDR) es un sistema de radiocomunicaciones donde la mayoría de las funciones de radio se implementan mediante software en lugar de hardware dedicado, permitiendo flexibilidad y reconfiguración dinámica. Herramientas como GNU Radio [?] facilitan la creación y evaluación de sistemas digitales mediante diagramas de flujo que ilustran etapas esenciales en el procesamiento de señales. GNU Radio constituye el marco de trabajo estándar para este propósito; no obstante, la construcción de sistemas avanzados exige trascender el uso de bloques predeterminados mediante la programación de módulos a la medida. Este informe aborda el desarrollo e integración de bloques nativos en Python para procesar flujos de datos en tiempo real. Para estructurar la solución, se codificaron algoritmos de acumulación y diferenciación discreta, seguidos por un bloque de estimación estadística orientado a caracterizar señales dinámicas. Dichos módulos se integraron en una aplicación práctica diseñada para la mitigación y análisis de ruido, demostrando la utilidad de los descrip-

tores estadísticos en el filtrado y detección de señales. Adicionalmente, el proceso de desarrollo e integración del código se coordinó de forma distribuida utilizando un flujo de trabajo basado en ramas en Git y GitHub.

II Metodología

El desarrollo de esta práctica se centró en el diseño, implementación y validación de tres algoritmos utilizando la herramienta GNU Radio Companion. El flujo de trabajo se dividió en tres fases principales correspondientes a cada bloque procesador: un acumulador, un diferenciador y un analizador estadístico. Los códigos del bloque de python de cada uno de los algoritmos se extrajeron del libro guía[?].

- Diseño del Bloque Acumulador: El primer algoritmo consistió en la implementación de un sistema acumulador discreto, cuya función matemática responde a la ecuación: $y[n] = y[n - 1] + x[n]$.
- Diseño del Bloque Diferenciador: El segundo algoritmo implementó un diferenciador discreto de primer orden, modelado mediante la ecuación $y[n] = x[n] - x[n - 1]$.
- Implementación del Analizador Estadístico: El tercer algoritmo implementa el cálculo de métricas estadísticas en tiempo real (Media, Media Cuadrática, Valor RMS, Potencia Promedio y Desviación Estándar).

III RESULTADOS Y ANÁLISIS DE DATOS

III-A Bloque Acumulador

Para analizar el comportamiento del bloque acumulador en GNU Radio, se implementó un diagrama de flujo compuesto por un bloque Vector Source, un bloque Throttle, un bloque acumulador denominado e_Acum y

dos bloques QT GUI Time Sink para visualizar simultáneamente las señales de entrada y salida.

- **Vector Source:** Funciona como el generador de señales de entrada al sistema. Modela un flujo de datos discreto a partir del vector periódico $[1, 1, 1, -1, -1, -1]$ habilitado con la opción *Repeat: Yes*. Esto produce una onda cuadrada simétrica que alterna entre valores lógicos de $+1$ y -1 .
- **e_Acum (Bloque Acumulador):** Es el módulo programado en Python que implementa la función de integración discreta en el tiempo mediante la ecuación de diferencias:

$$y[n] = y[n-1] + x[n]$$

El algoritmo mantiene una variable interna que acumula el valor de las muestras entrantes $x[n]$. Al recibir valores de $+1$, la salida $y[n]$ se incrementa linealmente formando una rampa ascendente; al recibir valores de -1 , la salida se decrementa generando una rampa descendente.

- **Throttle:** Actúa como limitador de velocidad del flujo de muestras en la CPU. Restringe la tasa de procesamiento a la frecuencia especificada por la variable `samp_rate` (32 kHz), asegurando una simulación coherente con el tiempo real.
- **QT GUI Time Sink (Entrada):** Visualizador osciloscópico conectado a la salida directa del *Vector Source*. Permite observar la señal de entrada cuadrada en el dominio del tiempo.
- **QT GUI Time Sink (Salida):** Visualizador conectado a la salida del bloque *e_Acum*. Muestra la señal procesada tras el proceso de acumulación.

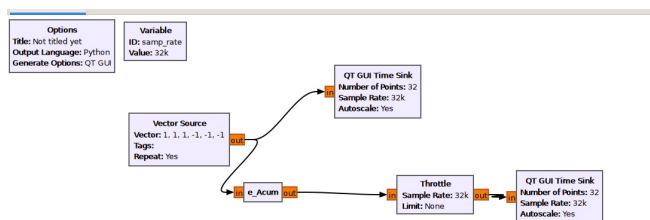


Figura 1: Diagrama de Bloques Acumulador

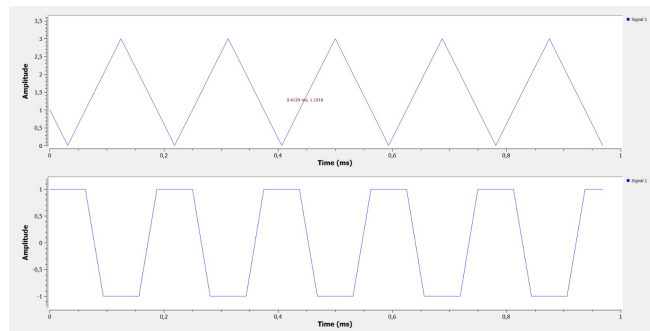


Figura 2: Resultado obtenido

Conclusión del Bloque Acumulador

La respuesta obtenida en la simulación confirma que el bloque *e_Acum* se comporta de manera equivalente a un **integrador discreto en el tiempo**. Como se observa al comparar las gráficas, el procesamiento de la onda cuadrada alternada entre $+1$ y -1 genera a la salida una onda triangular acotada en el rango $[0, 3]$. La acumulación sucesiva de tres muestras positivas consecutivas eleva el nivel de la señal desde 0 hasta su valor pico de 3, mientras que la llegada de tres muestras negativas produce la pendiente descendente hasta retornar a 0. Esto valida matemáticamente la correcta integración recursiva del algoritmo desarrollado en Python para el procesamiento de señales en tiempo real.

III-B Bloque Diferenciador

Para validar el comportamiento del bloque diferenciador en GNU Radio, se diseñó un esquema de simulación enfocado en el análisis de señales analógicas. Como señal de prueba se empleó una señal cosenoidal con una frecuencia fundamental de 1kHz y una frecuencia de muestreo de 32k.

El flujo de trabajo implementado consta de los siguientes bloques principales:

- **Signal Source:** Genera la señal de prueba cosenoidal de referencia con amplitud unitaria.
- **Throttle:** Controla la tasa de consumo de muestras para adecuar la simulación a la velocidad de procesamiento en tiempo real sin sobrecargar el procesador.
- **e_Diferenciador:** Bloque personalizado encargado de calcular la derivada numérica de la señal de entrada mediante operaciones discretas en el dominio temporal.
- **QT GUI Time Sink:** Osciloscopio virtual configurado con dos canales (entrada en azul y salida diferenciada en rojo) y una ventana optimizada a

128 puntos para facilitar la visualización detallada de los ciclos.

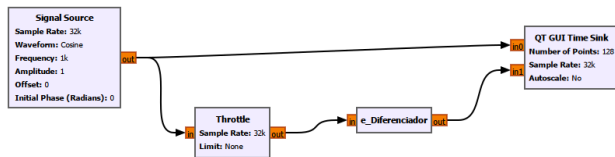


Figura 3: Diagrama de Bloques del diferenciador

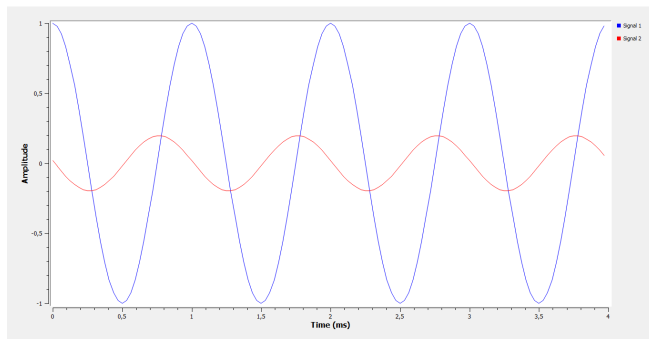


Figura 4: Resultado Obtenido para una señal cosenoidal

En la gráfica 5 en el dominio del tiempo se visualiza de forma simultánea la señal cosenoidal original (Canal 1 - Azul) y la respuesta entregada por el bloque diferenciador (Canal 2 - Rojo). Del comportamiento observado se destacan dos aspectos fundamentales: Relación de Fase y Polaridad: Matemáticamente, la derivada de una señal coseno ideal $x(t) = \cos(\omega t)$ corresponde a un seno invertido (es decir, $-\omega \sin(\omega t)$). En la simulación se comprueba que, ante los cruces por cero de la señal azul, la salida roja alcanza sus valores pico con la inversión de polaridad esperada para una diferenciación de primer orden. Escalamiento de Amplitud: La amplitud de la onda resultante muestra una reducción controlada por la ganancia discreta del algoritmo y la frecuencia de la señal, reflejando el comportamiento típico de un sistema diferenciador.

III-C Analizador Estadístico

Es un **Embedded Python Block** (bloque embebido en Python) de GNU Radio, implementado como una subclase de `gr.sync_block`. Recibe una entrada (la señal) y produce cinco salidas simultáneas, cada una con una estadística calculada de forma acumulativa desde el inicio de la simulación:

GNU Radio no procesa la señal muestra por muestra, sino en bloques (buffers) de tamaño variable N , cada vez que se invoca el método `work()`. Como las estadísticas deben ser acumulativas es decir, deben considerar todas las muestras procesadas desde el arranque y no solo el bloque actual, el código guarda el estado entre llamadas mediante variables de instancia inicializadas en el constructor. Sin esta persistencia, cada llamada a `work()` reiniciaría el cálculo desde cero y el resultado no representaría el promedio real de toda la señal procesada.

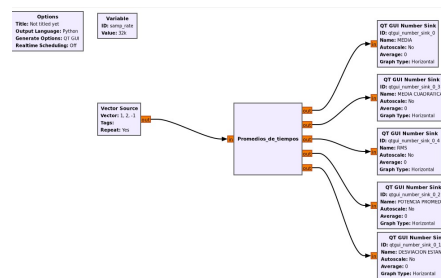


Figura 5: Diagrama de Bloque Estadístico

Fórmulas de cálculo

Cada salida del bloque corresponde a una estadística distinta, calculada según las expresiones mostradas en la Tabla 1.

Tabla 1: Fórmulas de cálculo de cada estadística

Estadística	Fórmula
Media	$\bar{x}[n] = \frac{1}{n} \sum_{k=1}^n x[k]$
Media Cuadrática	$\overline{x^2}[n] = \frac{1}{n} \sum_{k=1}^n x[k]^2$
RMS	$x_{RMS} = \sqrt{\overline{x^2}}$
Potencia Promedio	$P_{prom} = x^2$
Desviación Estándar	$\sigma = \sqrt{\frac{1}{n} \sum_{k=1}^n (x[k] - \bar{x})^2}$

Validación de resultados

Para verificar el correcto funcionamiento del bloque se utilizó como señal de prueba el vector (1, 2, -1), repetido periódicamente. La Tabla ?? compara los valores obtenidos analíticamente con los arrojados por la simulación en GNU Radio.

Como se observa, los valores simulados coinciden con los valores teóricos calculados analíticamente, con diferencias mínimas atribuibles al redondeo de punto flotante y al número finito de muestras acumuladas durante la simulación, validando así el correcto funcionamiento del bloque.

Tabla 2: Comparación entre valores teóricos y simulados
(vector de prueba: 1, 2, -1)

Estadística	Valor teórico	Valor simulado (GNU Radio)
Media	0.667	0.6666
Media Cuadrática	2.000	2.0003
RMS	1.414	1.4143
Potencia Promedio	2.000	2.0003
Desviación Estándar	1.247	1.2480

IV Conclusiones

- Se comprobó que el bloque acumulador `e_Acum` actúa como un integrador discreto, transformando la señal cuadrada de entrada $[1, 1, 1, -1, -1, -1]$ en una salida tipo rampa/triangular acotada en $[0, 3]$, evidenciando su naturaleza recursiva y validando la ecuación $y[n] = y[n-1] + x[n]$.
- Se verificó que el bloque diferenciador `e_Diferenciador` identifica con precisión los flancos y transiciones de una señal cosenoidal, generando picos de magnitud proporcional al cambio de nivel e invirtiendo la polaridad en los cruces por cero, validando así la ecuación $y[n] = x[n] - x[n-1]$.
- El analizador estadístico calculó correctamente las cinco métricas (Media, Media Cuadrática, RMS, Potencia Promedio y Desviación Estándar) en tiempo real, con resultados prácticamente idénticos a los valores teóricos para el vector de prueba $(1, 2, -1)$, confirmando que estas métricas dependen de los valores de la señal y no de la velocidad de procesamiento ni de la tasa de muestreo.
- Se determinó que la precisión de la media es sensible al número de muestras acumuladas, mientras que los cambios en la energía de la señal se reflejan de forma más inmediata en el valor RMS, validando la sensibilidad de este último para la caracterización de señales estocásticas.
- El uso correcto de variables de instancia para preservar el estado entre llamadas al método `work()` resultó indispensable, dado que GNU Radio procesa la señal por bloques (buffers) de muestras y no de forma continua.
- La implementación de estas métricas en tiempo real resulta aplicable en escenarios reales de procesamiento de audio, como la detección de desplazamientos DC mediante la media o la medición de sonoridad percibida (*loudness*) a través del RMS, parámetros útiles para la optimización de mezcla y diseño de compresores.